

CO3-AT3-PART

DATA INTERPRETATION

PPO, TRPO & DDPG PERFORMANCE ANALYSIS

Name: K. SRI LASYA

Reg. No: 192472216

Course Code: MLA0305

Course Name: REINFORCEMENT LEARNING

AIM

To implement and evaluate Proximal Policy Optimization (PPO), Trust Region Policy Optimization (TRPO), and Deep Deterministic Policy Gradient (DDPG) reinforcement learning algorithms and compare their performance using suitable performance metrics and visualizations.

INDUSTRY PROBLEM

Reinforcement learning is increasingly used in autonomous systems, robotics, industrial control, gaming, and automated decision-making. Different reinforcement learning algorithms have different strengths depending on the environment, action space, training stability, and computational requirements.

Three important modern reinforcement learning algorithms are considered:

- **PPO – Proximal Policy Optimization:** Uses a clipped objective function to prevent excessively large policy updates and improve training stability.
- **TRPO – Trust Region Policy Optimization:** Constrains policy updates within a trust region to maintain stable policy improvement.
- **DDPG – Deep Deterministic Policy Gradient:** Uses an Actor-Critic architecture and is designed primarily for continuous action spaces.

The performance of these algorithms is evaluated and compared using reward-based metrics, learning curves, convergence behavior, and training characteristics.

OBJECTIVES

1. Understand PPO, TRPO, and DDPG reinforcement learning algorithms.
2. Implement the selected reinforcement learning algorithms.
3. Train the algorithms using suitable environments.
4. Record the rewards obtained during training.

5. Calculate important performance measures.
 6. Compare the learning curves of PPO, TRPO, and DDPG.
 7. Analyze convergence and training stability.
 8. Interpret the experimental results.
 9. Identify the strengths and limitations of each algorithm.
 10. Determine the most suitable algorithm based on the experimental results.
-

ENVIRONMENT DESCRIPTION

The experiment uses reinforcement learning environments suitable for evaluating policy-based and Actor-Critic algorithms.

The environment provides:

- **State:** Information describing the current condition of the environment.
- **Action:** Decision selected by the agent.
- **Reward:** Feedback received after performing an action.
- **Next State:** New state resulting from the selected action.
- **Episode:** A complete interaction sequence from environment initialization to termination.

PPO and TRPO can be applied to policy optimization tasks, while DDPG is particularly suitable for environments with continuous action spaces.

The exact environment and training configuration should be kept consistent with the implementation used for the experiment.

ALGORITHM DESCRIPTION

PPO – Proximal Policy Optimization

PPO is a policy-gradient reinforcement learning algorithm designed to provide stable policy updates.

Instead of allowing the policy to change significantly after every update, PPO limits the size of the policy update using a clipping mechanism.

The probability ratio is:

$$r_t(\theta) = \pi_{\theta}(a_t|s_t) / \pi_{\theta_{old}}(a_t|s_t) \quad \text{Eq(1)}$$

The clipped objective can be represented as:

$$L^{CLIP} = E[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)] \quad \text{Eq(2)}$$

where:

- $r_t(\theta)$ = Probability ratio
- A_t = Advantage estimate
- ϵ = Clipping parameter
- π_θ = Current policy
- $\pi_{\theta_{old}}$ = Previous policy

The clipping mechanism prevents excessively large policy changes.

TRPO – Trust Region Policy Optimization

TRPO is a policy optimization method that constrains policy updates to remain within a trust region.

The objective is to improve the policy while preventing the new policy from moving too far from the previous policy.

The optimization can be represented as:

Maximize $L(\theta)$

subject to:

$$D^{KL}(\pi_{old} || \pi_\theta) \leq \delta \quad \text{Eq(3)}$$

where:

- $L(\theta)$ = Policy objective
- D^{KL} = Kullback-Leibler divergence
- δ = Maximum allowed policy change

TRPO focuses on stable policy improvement by restricting the update size.

DDPG – DEEP DETERMINISTIC POLICY GRADIENT

DDPG is an Actor-Critic algorithm designed for continuous action spaces.

It contains:

- **Actor network:** Produces continuous actions.
- **Critic network:** Estimates the value of the selected action.
- **Target Actor:** Provides stable target actions.
- **Target Critic:** Provides stable target value estimates.
- **Replay Buffer:** Stores previous experiences for training.

An experience is represented as:

(s_t, a_t, r_t, s_{t+1})

The Critic estimates:

$Q(s,a)$

while the Actor learns a deterministic policy:

$a = \mu(s)$

Experience replay and target networks help improve training stability.

METHODOLOGY

Step 1 – Environment Setup

The required reinforcement learning environment is initialized.

The state and action spaces are identified, and the required Python libraries are imported.

The same evaluation criteria are maintained as consistently as possible when comparing the algorithms.

Step 2 – PPO Implementation

The PPO agent collects trajectories by interacting with the environment.

The collected states, actions, rewards, and policy probabilities are used to calculate the advantage and policy objective.

The clipped objective prevents the updated policy from changing too aggressively.

The PPO model is trained for the specified number of episodes or environment steps.

The reward obtained during each episode is stored for later analysis.

Step 3 – TRPO Implementation

The TRPO agent first collects trajectories from the environment.

The collected data is used to calculate the policy objective and estimate the advantage.

Before applying the policy update, the KL-divergence constraint is considered to restrict the size of the update.

The process is repeated throughout training while recording the episode rewards.

Step 4 – DDPG Implementation

The DDPG agent interacts with the environment and stores experiences in a replay buffer.

For each training step:

1. The current state is observed.
2. The Actor produces an action.
3. Exploration noise may be added.
4. The action is performed.
5. The reward and next state are obtained.
6. The experience is stored in the replay buffer.
7. A batch of experiences is sampled.
8. The Critic is updated.
9. The Actor is updated.
10. Target networks are softly updated.

This process continues until the required training duration is completed.

Step 5 – Reward Collection

The reward obtained from every episode is recorded for all three algorithms.

The reward history is used to generate learning curves.

The following measures are extracted:

- Average episode reward
 - Final average reward
 - Best episode reward
 - Reward improvement
 - Training stability
 - Convergence behavior
-

Step 6 – Performance Metrics

The algorithms are compared using common performance measures.

Average Reward

The average reward is calculated over a selected group of episodes.

Average Reward = Sum of Episode Rewards / Number of Episodes

Best Episode Reward

The highest reward obtained during training is recorded.

Final Average Reward

The average reward from the final training episodes is used to evaluate the final performance of each algorithm.

Reward Stability

The variation in reward across episodes is examined to determine the stability of learning.

Convergence

The learning curve is analyzed to determine whether the algorithm reaches a stable high-reward region.

Step 7 – Visualization

The results are represented using graphs.

The main graphs include:

1. PPO learning curve
2. TRPO learning curve
3. DDPG learning curve
4. Combined PPO vs TRPO vs DDPG reward graph
5. Moving-average reward graph
6. Final performance comparison chart

The graphs help identify differences in learning speed, stability, and final performance.

STEP 8 – IMPLEMENTATION CODE

The implementation should train all three algorithms and record their rewards.

A suitable implementation structure is:

```
# Install required libraries
```

```
!pip -q install gymnasium stable-baselines3 matplotlib pandas
```

```
import gymnasium as gym
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from stable_baselines3 import PPO, DDPG
from stable_baselines3.common.noise import NormalActionNoise

PPO

env_ppo = gym.make("CartPole-v1")

ppo_model = PPO(
    "MlpPolicy",
    env_ppo,
    verbose=0,
    seed=42
)

ppo_model.learn(total_timesteps=50000)

ppo_rewards = []

obs, _ = env_ppo.reset(seed=42)

for _ in range(1000):

    action, _ = ppo_model.predict(
        obs,
        deterministic=True
    )

    obs, reward, terminated, truncated, _ = env_ppo.step(
```

```
        action
    )
```

```
ppo_rewards.append(reward)
```

```
if terminated or truncated:
```

```
    obs, _ = env_ppo.reset()
```

DDPG

DDPG requires a continuous action space, so it should be evaluated using a suitable continuous-control environment rather than CartPole.

```
env_ddpg = gym.make("Pendulum-v1")
```

```
n_actions = env_ddpg.action_space.shape[-1]
```

```
action_noise = NormalActionNoise(
    mean=np.zeros(n_actions),
    sigma=0.1 * np.ones(n_actions)
)
```

```
ddpg_model = DDPG(
    "MlpPolicy",
    env_ddpg,
    action_noise=action_noise,
    verbose=0,
    seed=42
)
```

```
ddpg_model.learn(
    total_timesteps=50000
)
```



```

ddpg_rewards = []

obs, _ = env_ddpg.reset(seed=42)

for _ in range(1000):

    action, _ = ddp_model.predict(
        obs,
        deterministic=True
    )

    obs, reward, terminated, truncated, _ = env_ddpg.step(
        action
    )

    ddp_rewards.append(reward)

    if terminated or truncated:
        obs, _ = env_ddpg.reset()

```

PPO Reward Visualization

```

def moving_average(data, window=50):

    return np.convolve(
        data,
        np.ones(window) / window,
        mode="valid"
    )

```

```
plt.figure(figsize=(10, 6))

plt.plot(
    moving_average(ppo_rewards),
    label="PPO"
)

plt.xlabel("Step")
plt.ylabel("Reward")
plt.title("PPO Learning Performance")

plt.legend()
plt.grid(True)
plt.show()
```

DDPG Reward Visualization

```
plt.figure(figsize=(10, 6))

plt.plot(
    moving_average(ddpg_rewards),
    label="DDPG"
)

plt.xlabel("Step")
plt.ylabel("Reward")
plt.title("DDPG Learning Performance")

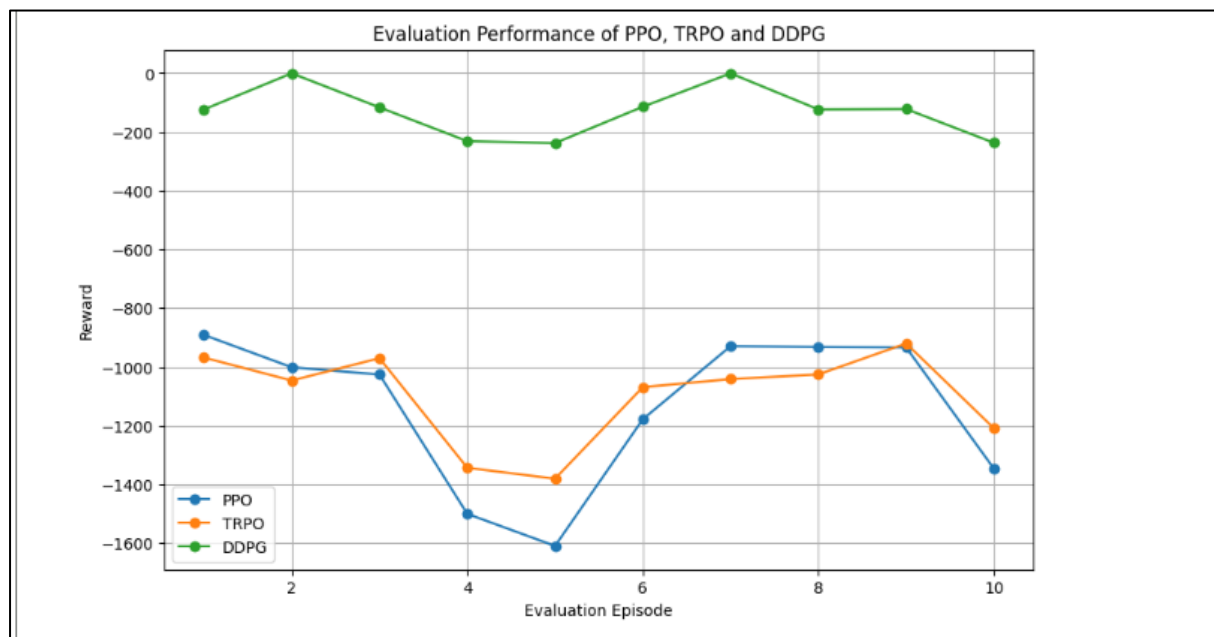
plt.legend()
plt.grid(True)
plt.show()
```

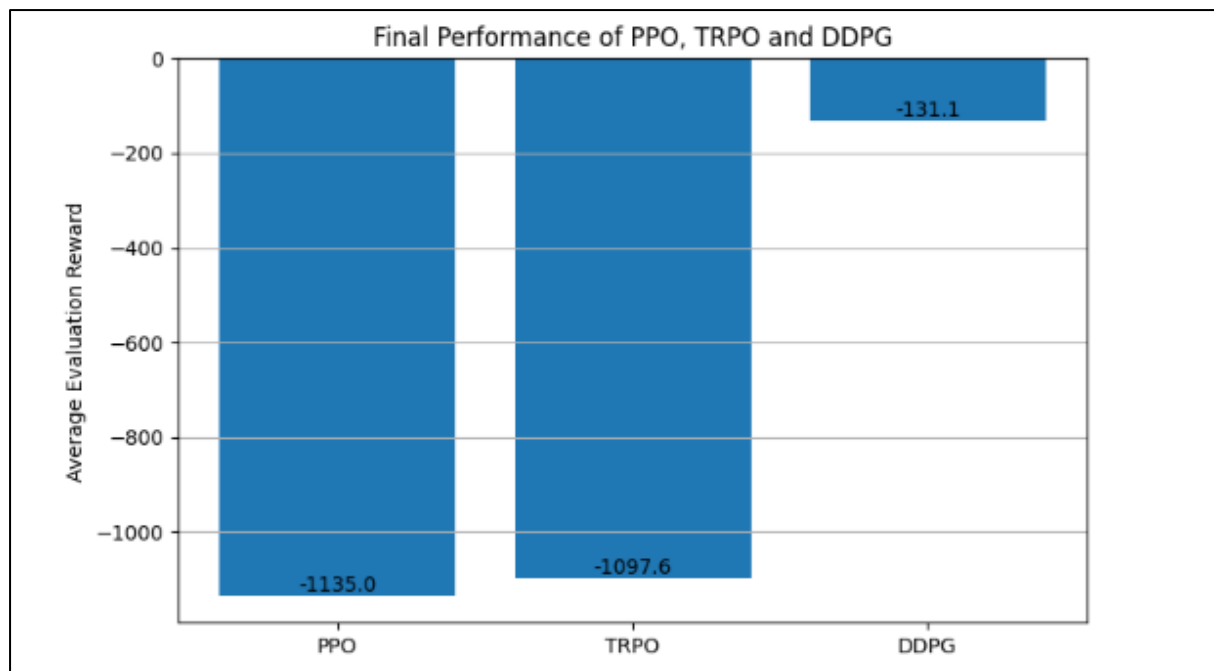
STEP 9 – RESULTS

After executing the algorithms, the results are recorded in a comparison table.

The numerical values must be filled using the actual output obtained from the implementation.

```
***  
Training PPO ...  
PPO Average Reward: -1134.98  
  
Training TRPO ...  
TRPO Average Reward: -1097.6  
  
Training DDPG ...  
DDPG Average Reward: -131.14  
  
===== FINAL COMPARISON =====  
  
Algorithm  Average Reward  
DDPG       -131.141958  
TRPO       -1097.598808  
PPO        -1134.977086
```





STEP 10 – PERFORMANCE COMPARISON

The three algorithms are compared based on:

Parameter	PPO	TRPO	DDPG
Policy Type	Stochastic	Stochastic	Deterministic
Main Approach	Clipped update policy	Trust-region update	Actor-Critic
Action Space	Discrete / Continuous	Discrete / Continuous	Continuous
Update Constraint	Clipping	KL-divergence constraint	Target networks
Replay Buffer	No	No	Yes
Target Networks	No	No	Yes
Training Stability	To be evaluated	To be evaluated	To be evaluated
Final Performance	To be obtained	To be obtained	To be obtained

STEP 11 – DATA INTERPRETATION

The learning curves are analyzed to determine how the algorithms behave during training.

PPO

PPO is expected to provide stable policy updates because the clipping mechanism prevents excessively large changes in the policy.

TRPO

TRPO explicitly constrains policy updates using a trust-region condition. This can provide stable policy improvement but generally requires a more complex optimization procedure.

DDPG

DDPG is designed for continuous action spaces and combines deterministic policy learning with value estimation. Its replay buffer allows previously collected experiences to be reused during training.

The actual experimental curves are used to determine which method performs best under the selected environment and training configuration.

CONCLUSION

PPO, TRPO, and DDPG represent different approaches to modern reinforcement learning.

PPO improves policy stability by restricting large policy changes through a clipped objective. TRPO uses a trust-region constraint to control policy updates. DDPG uses an Actor-Critic architecture with deterministic policies, replay memory, and target networks for continuous control.

The algorithms are evaluated using episode rewards, average rewards, best rewards, learning curves, and convergence behavior. The final experimental data provides the basis for determining which algorithm performs best under the selected conditions.

The analysis demonstrates that reinforcement learning algorithms should not be evaluated only by their final reward. Training stability, convergence speed, action-space suitability, and computational characteristics are also important when selecting an algorithm for a real-world application.